Software Architecture and Design Specification

Project: Charity Event Fundraiser Tracking System

Version: 1.0

Authors: Sujoy Sen – PES2UG23CS621

Ishaan Sinha – PES2UG23CS921

Ishika Raj – PES2UG23CS923

Tanishq Raj – PES2UG23CS642

---

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the **Charity Event Fundraiser Tracking System**. It serves as a blueprint for the implementation of the system.

1.2 Scope

The system's scope covers event creation, donor pledge management, progress tracking, user access control, reporting, and security.

Excluded items: Third-party payment gateway processing and external NGO integrations.

1.3 Audience

The audience for this document includes developers, QA engineers, security auditors, and maintenance teams.

1.4 Definitions

- **Pledge:** A donor's commitment (financial or otherwise) to a fundraising event.
- **RTM:** Requirements Traceability Matrix.
- **SRS:** Software Requirements Specification.
- **CFT:** Charity Event Fundraiser Tracking System.

---

2. Architecture

2.1 Goals & Constraints

Goals:

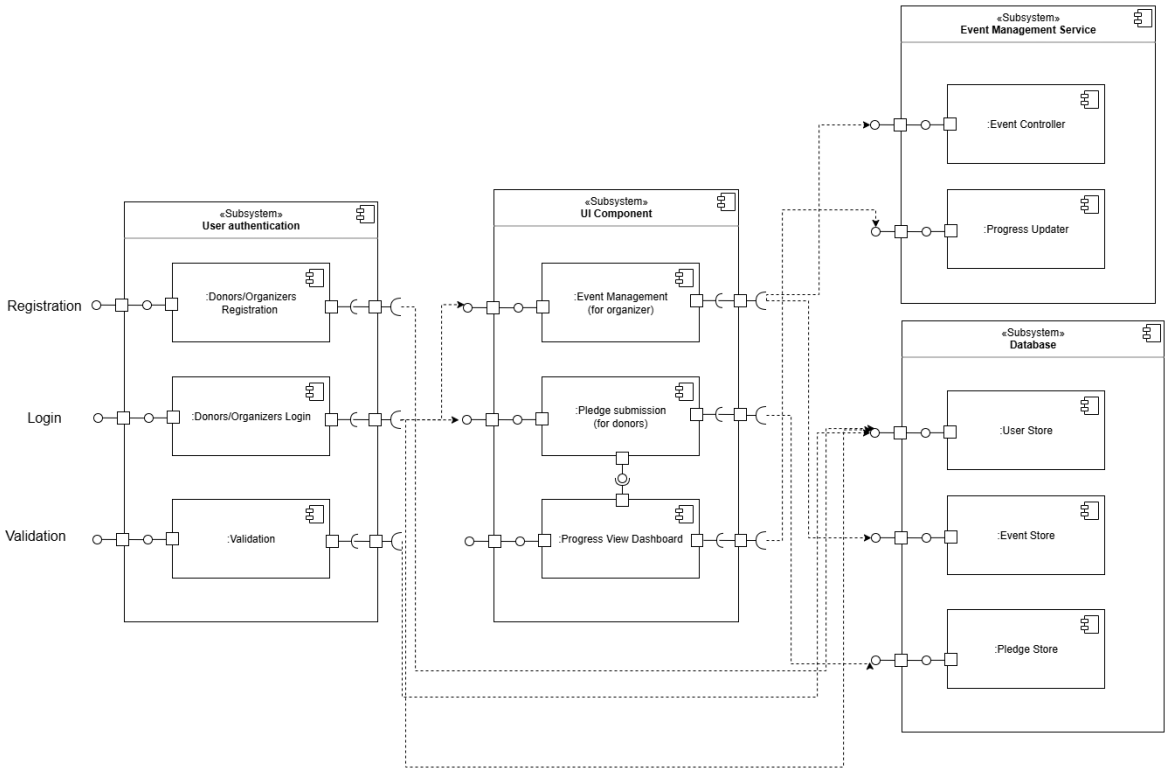
- To ensure **real-time tracking** of fundraising goals.
- To provide **transparency** for donors.
- To achieve **operational efficiency** for event organizers.

Constraints:

- Compliance with **Data Security Standards** (e.g., GDPR, PCI-DSS).

2.2 Component (UML) Diagram

The system is structured into a logical set of components for clear separation of concerns, typical of an N-Tier architecture.



2.3 Component Descriptions

- **Web/Mobile UI:** Handles all organizer and donor interaction, presenting event information, and facilitating pledge submission.
- **API Gateway/Web Server:** Acts as the single entry point for all client requests, routing them to the appropriate backend service.
- **User/Auth Service:** Handles organizer login (CTF-F-010), secure password management (CTF-F-011), and access control (e.g., admin revocation CTF-F-012).
- **Event Management Service:** Processes event creation (CFT-F-001), editing/deletion (CFT-F-002), and closure (CFT-F-003).
- **Pledge/Donation Service:** Manages donor pledge submission (CTF-F-004) and validation (CTF-F-005).
- **Progress Tracking Service:** Aggregates pledges (CFT-F-007) and calculates and updates the real-time progress against the event target (CTF-F-006, CFT-F-008).

- **Reporting Service:** Generates reports, including visual reports (CFT-F-009) and exportable lists (CFT-F-013).
- **Database:** Central storage for all event, donor, pledge, and user data.

2.4 Chosen Architecture Pattern and Rationale

Pattern: Layered Architecture (N-Tier)

Rationale: This pattern is chosen for its simplicity, clear separation of concerns (Presentation, Business Logic, Data), and maintainability, which is suitable for a system with distinct, manageable features like Event Management and Pledge Tracking.

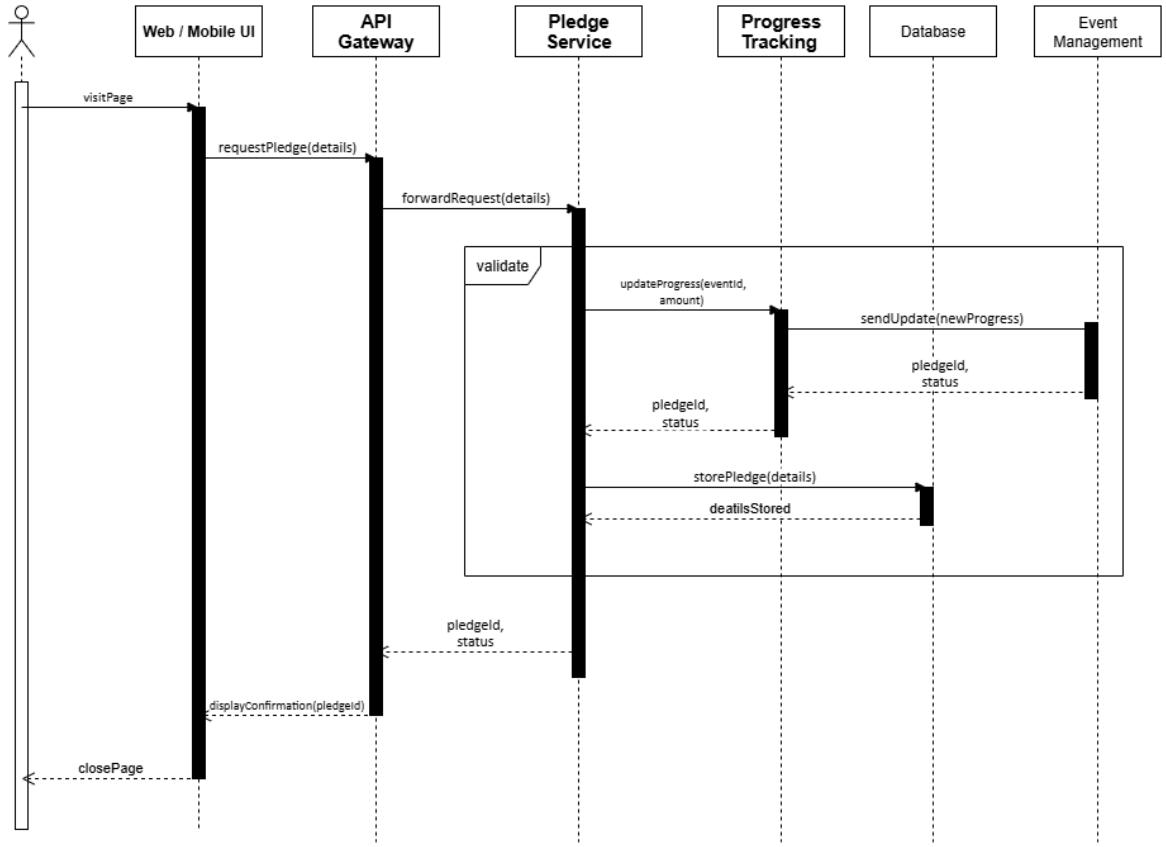
2.5 Technology Stack & Data Stores

- **Frontend: Next.js**
- **Backend: Node.js/Express.js (for APIs)**
- **Database: MongoDB (for flexibility)**
- **Data Security: TLS (via Next.js/Node.js server config)**

3. Design

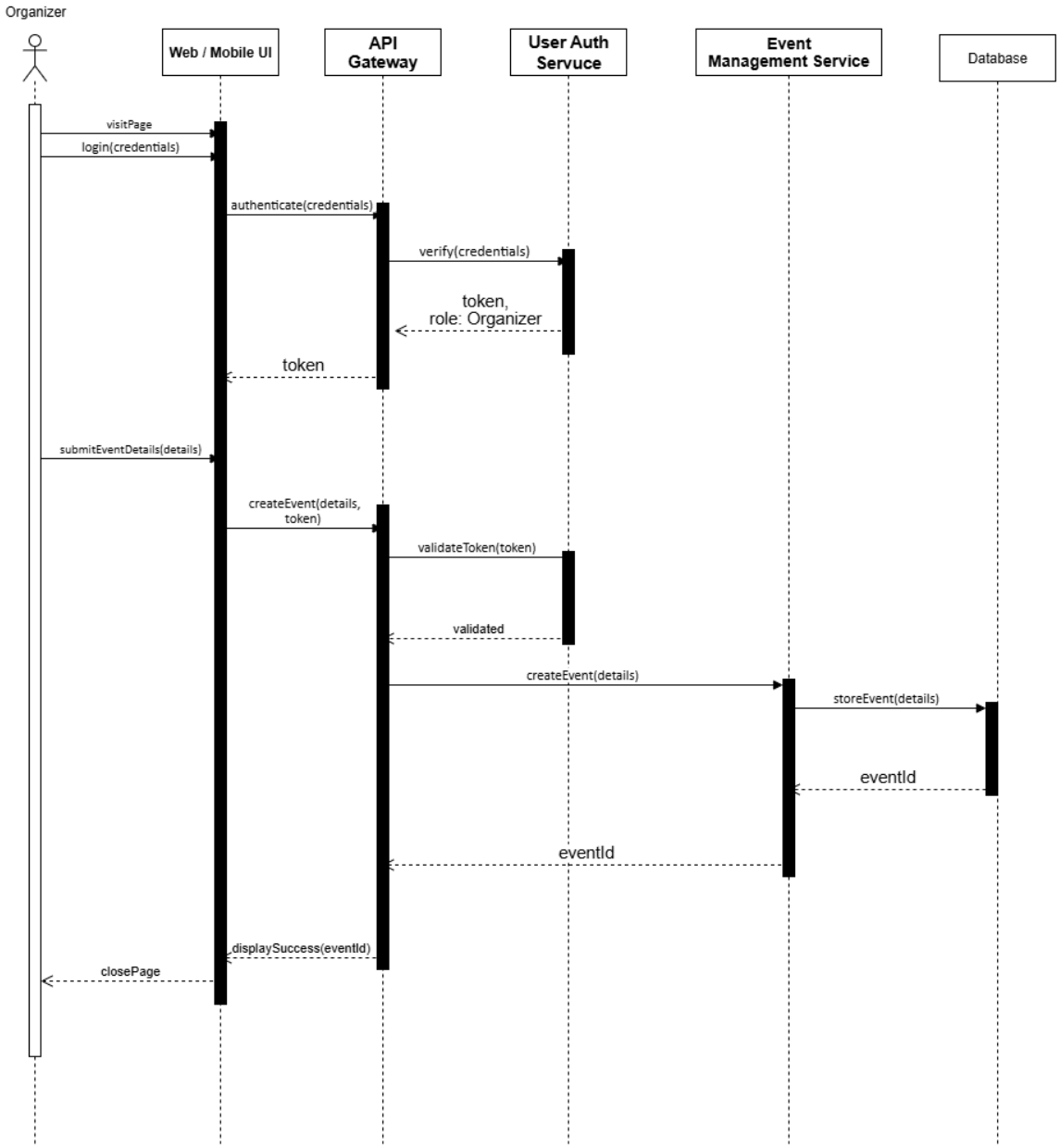
3.1 UML Sequence Diagrams

Sequence Diagram 1: Donor Pledge Submission (CFT-F-004)



Object/Actor	Action
Donor	submitPledge(details)
Web/Mobile UI	requestPledge(details) → **API Gateway**
API Gateway	forwardRequest(details) → **Pledge Service**
Pledge Service	storePledge(details) → **Database**
Pledge Service	updateProgress(eventId, amount) → **Progress Tracking Service**
Progress Tracking Service	sendUpdate(newProgress) → **Event Management Service**
Pledge Service	← pledgeId, status
API Gateway	← pledgeId, status
Web/Mobile UI	displayConfirmation(pledgeId)

Sequence Diagram 2: Organizer Event Creation (CFT-F-001)



Object/Actor	Action
Organizer	login(credentials)
Web UI	authenticate(credentials) → **API Gateway**
API Gateway	verify(credentials) → **User/Auth Service**
User/Auth Service	← token, role: Organizer
Web UI	← token

Object/Actor	Action
Organizer	submitEventDetails(details)
Web UI	createEvent(details, token) → **API Gateway**
API Gateway	validateToken(token) → **User/Auth Service**
API Gateway	createEvent(details) → **Event Management Service**
Event Management Service	storeEvent(details) → **Database**
Event Management Service	← eventId
API Gateway	← eventId
Web UI	displaySuccess(eventId)

3.2 API Design

Component	Endpoint	Method	Purpose	Request Body	Response Body	Errors
Event Management	/events	POST	Create fundraising event (CFT-F-001)	{eventName, targetGoal, startDate, organizerId}	{eventId, status: "Created"}	400 Invalid Input, 403 Unauthorized
Pledge/Donation	/pledges	POST	Submit donor pledge (CTF-F-004)	{eventId, donorName, email, pledgeAmount}	{pledgeId, status: "Pending"}	404 Event Not Found, 400 Invalid Amount

3.3 Security Architecture

The system employs the following to meet security goals and compliance requirements (GDPR, PCI-DSS, MFA):

- **Authentication:** Multi-Factor Authentication (MFA) is required for Organizer and Admin roles.
- **Communication Security:** All data in transit is protected using **TLS** encryption.
- **Authorization:** Role-Based Access Control (RBAC) is enforced to ensure organizers can only manage their own events and administrators can revoke user access immediately (CTF-F-012).

3.4 UX Design

The UX design prioritizes two main user groups:

- **Organizers:** An intuitive dashboard for managing events and viewing visual reports (pie, bar graphs) for donation trends (CFT-F-009).
- **Donors:** A seamless, trustworthy interface for pledge submission, clearly displaying the event's **real-time progress bar** (CFT-F-008) against its target goal.

3.5 Error Handling, Logging & Monitoring

- **Error Handling:** A standardized set of HTTP status codes and error messages will be used (e.g., 401 for Invalid Authentication, 404 for Not Found).
- **Logging:** Logging will be enabled for auditing and troubleshooting, but no sensitive donor or pledge information will be stored in plain text logs.
- **Monitoring:** Key metrics will be monitored, including: Pledge submission rate, event creation success rate, real-time goal progress updates, and component uptime.